

LAB 2 결과보고서

순차논리회로의 FPGA 구현과 검증

과목	전기전자컴퓨터설계실험 2
이름 · 학번	김태이 · 2023440158
작성일	2026-09-28
대상 소자	AMD Spartan-7 xc7s75fgga484-1 (HBE-Combo II-DLD)
검증 도구	VS Code + Icarus Verilog, Vivado 2026.1 XSim
보드 시현	Lab 2.08 8자리 7세그먼트 자리 스캔
GitHub · 커밋	github.com/tae2yi/UOS_ece2 · 27eefdb
시현 영상	youtube.com/playlist?list=PLbHILnDMwa6Y (ece2-lab2 재생목록)

요약 업/다운 카운터, 클록 분주기, 레지스터 쌍, 시프트 레지스터, PISO, Moore-Mealy 상태 머신, 8자리 7세그먼트 자리 스캔의 8개 순차논리회로를 Verilog로 작성하고, 같은 RTL과 자기검사 테스트벤치를 VS Code의 Icarus와 Vivado XSim에서 실행하였다. 8개 회로 모두 Icarus에서 LAB2_PASS를 얻었고, Vivado 파형에서도 리셋·enable·상태 전이·출력 시점이 같은 값으로 관찰되었다. 교안의 수정실험 8건은 모두 예측한 검사와 시각에서 실패했으며 복구 후 다시 통과하였다. Lab 2.08은 1 kHz 보드 클록으로 합성·구현하여 setup·hold 위반 없이 bit 파일을 만들었고, HBE-Combo II-DLD 보드에서 DIP 스위치 조작에 따라 첫 자리가 0→1→3→7로 바뀌는 것을 확인하였다. 이 표시는 시뮬레이션의 세그먼트 코드 fc·60·f2·e0과 일치한다.

1 실험 목적과 보고 범위

본 실험의 목적은 클록 에지에서 저장값이 갱신되는 순차논리회로의 동작을 파형으로 설명하고, 같은 설계를 두 시뮬레이터와 실제 FPGA 보드에서 단계적으로 검증하는 것이다. 조합회로를 다룬 Lab 1과 달리 Lab 2의 출력은 현재 입력뿐 아니라 이전 상태에 의존하므로, 입력이 바뀐 순간과 다음 상승 에지에서 상태가 바뀐 순간을 구분해 읽는 것이 핵심이다 [1].

교안 00 START는 한 회로를 끝내는 순서를 RTL·TB 입력, VS Code 시뮬레이션, 정상·변경·복구 로그 정리, Vivado 등록과 bit 생성, 실제 보드 관찰의 다섯 단계로 제시한다 [1, p.5]. 본 보고서는 실험 후 레포트 요구사항 [1, p.25]을 표 1과 같이 대응시켜 작성하였다.

표 1. 실험 후 레포트 요구사항과 대응 위치 [1, p.25].

교안 요구사항	본 보고서 대응
Vivado 기능 시뮬레이션과 VS Code 결과 비교	3장: 8개 회로의 두 파형과 해석, 표 4
정상·변경·복구 결과 정리	4장: 수정실험 8건, 표 5
합성·구현 결과, 경고 해석, 핀·타이밍 조건, bit 파일	5장: 표 6~7, 경고 해석
실제 입출력 사진·영상	6장: 보드 사진 4장, 시현 영상 링크
장치 프로그램 화면	보존된 캡처 없음(미첨부)
GitHub 소스·레포트·관찰 자료 연결	9장: 저장소, 기준 커밋, 영상

2 설계 및 검증 방법

2.1 공통 동작 규칙

모든 테스트벤치는 10 ns 주기의 clk를 사용하며 상승 에지는 5, 15, 25, ... ns에 온다. TB는 step 태스크에서 상승 에지를 기다린 뒤 1 ns 후 결과를 비교한다. nonblocking(<=) 갱신이 끝나기 전의 값을 잘못 검사하지 않기 위해서다 [1, p.9]. 따라서 파형에서 입력은 6, 16, 26 ns처럼 에지 직후에 바뀌고, 상태는 그 다음 에지에서 바뀐다.

표 2. 동기 리셋과 enable의 우선순위 [1, p.10].

조건	다음 상승 에지의 동작
rst = 1	초기값으로 변경(동기 리셋, 최우선)
rst = 0, enable = 1	회로가 정한 상태로 갱신
rst = 0, enable = 0	이전 상태 유지

두 레지스터가 같은 에지에서 값을 주고받으면, nonblocking 대입에 의해 받는 쪽은 갱신 전의 값을 받는다 [1, p.11]. 이 규칙은 레지스터 쌍(3.3절)과 시프트 레지스터(3.4절)의 파형에서 직접 확인된다. 보드 입력력은 input_frontend에서 2단 플립플롭으로 동기화하고 버튼은 20 클록 동안 안정되어야 한 번의 펄스로 인정한다 [1, p.12].

2.2 회로별 설계와 검사 범위

표 3. 회로별 모듈, 핵심 동작과 Icarus 자기검사 결과. 검사 수와 종료 시각은 정상 실행의 simulation.log 기준이다.

일 습	모 듈	핵심 동작	검 사	종 료
01	counter4	down에 따른 증감, 15↔0 순환	36	356 ns
02	clock_divider	DIVISOR=10:2 분주, tick	129	436 ns
03	register_pair	load-transfer, 동시 제어	7	66 ns
04	shift_register4	bit 3 입력, LSB 방향 이동	8	106 ns
05	piso4	load 우선, MSB 먼저 직렬 출력	114	976 ns
06	moore_cycle	S0→S1→S2→S0, 출력=상태	23	226 ns
07	mealy_toggle	입력 1에서 토글, 출력=f(상태, 입력)	11	67 ns
08	segment_scan 8	8자리 순환, blank, 0-F 디코드	194	1306 ns

2.3 검증 절차

- **VS Code(Icarus):** 02 Simulate로 컴파일·실행하여 LAB2_PASS <모듈> checks=N과 종료 시각을 확인하고, VaporView로 wave.vcd를 열어 제어 입력과 출력 파형을 캡처하였다.
- **Vivado(XSim):** 같은 RTL과 TB를 등록하고 Run Behavioral Simulation으로 파형을 확인하였다. TB의 checks 변수를 파형 창에 추가해 검사 진행 수를 함께 기록하였다.

3

1/10)의 50% 듀티 파형이다. tick은 count=9인 95~105 ns 한 클럭 동안만 1이므로, 105 ns 에지에서 다른 회로가 enable로 한 번 소비할 수 있다. pulses가 105, 205, 305 ns 에지에서 1, 2, 3으로 증가하는 것은 30 클럭에 tick이 정확히 3회 소비되었음을 보여 준다. div2와 tick2는 매 클럭 토글하는 20 ns 주기 파형이다. 376~386 ns의 rst 펄스로 주기 중간에 리셋되면 divided는 0에서 다시 시작하여 435 ns 에지(리셋 후 5번째)에서 처음 1이 되며, 이 시점에서 시뮬레이션이 끝난다.

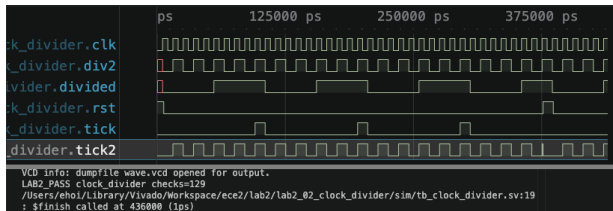


그림 4. Lab 2.02 VS Code 파형과 Icarus 로그. LAB2_PASS clock_divider checks=129, \$finish 436000 ps.

비교. VS Code 파형에서도 tick 펄스가 divided 하강 직전에 한 클럭 폭으로 나타나고, rst 재인가 뒤 divided가 low 반주기부터 다시 시작한다. 분주 출력 divided는 관찰용이며, 실제 회로는 tick을 공통 clk의 enable로 쓰도록 설계되어 있다 [1, p.13].

3.3 Lab 2.03 레지스터 저장과 이동

load만 1이면 stored에 입력을 저장하고, transfer만 1이면 value에 stored를 전달하며, 둘 다 1이면 value는 이전 stored를 받고, 둘 다 0이면 두 값을 유지해야 한다 [1, p.16].

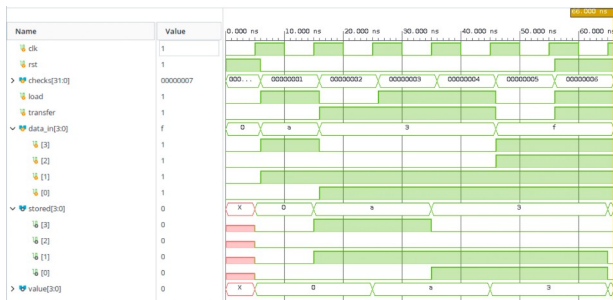


그림 5. Lab 2.03 Vivado XSim 파형. load-transfer, data_in, stored, value와 checks=7.

해석. 15 ns 에지에서 load=1이므로 stored=a가 되지만 value는 0으로 남는다(load는 전달하지 않음). 16 ns에 data_in이 3으로 바뀌고 transfer가 켜지면 25 ns

에지에서 value는 입력 3이 아니라 저장된 a를 받는다. 26 ns에 load도 켜진 뒤 35 ns 에지에서는 stored가 3으로, value는 같은 에지의 갱신 전 stored인 a를 받아 {stored,value}=3a가 된다. 45 ns 에지에서야 value가 3이 된다. 파형의 stored 3과 value 3 사이의 한 클럭 지연이 nonblocking 전달 규칙 [1, p.11]의 직접적인 증거이다. 46 ns에 data_in=f로 바뀌어도 load-transfer가 0이므로 두 값은 유지되고, 마지막 rst는 두 제어보다 우선하여 65 ns에 모두 0으로 만든다.

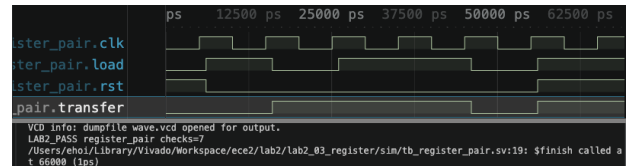


그림 6. Lab 2.03 VS Code 파형과 Icarus 로그. LAB2_PASS register_pair checks=7, \$finish 66000 ps.

비교. VS Code 캡처는 clk-load-rst-transfer 제어 파형을 보여 주며, load 두 구간과 transfer 구간의 겹침, 마지막 rst 동시 인가 위치가 Vivado와 같다. 저장값 버스는 Vivado 파형에서 확인하였다.

3.4 Lab 2.04 시프트 레지스터

새 입력은 bit 3으로 들어오고 기존 값은 bit 0 방향으로 이동한다. 입력을 1, 0, 1, 0 순서로 넣으면 저장값은 1000, 0100, 1010, 0101이 되어야 한다 [1, p.17].



그림 7. Lab 2.04 Vivado XSim 파형. enable-serial_in과 value[3:0] 및 각 비트, checks=8.

해석. value는 X → 0 → 8(1000) → 4(0100) → a(1010) → 5(0101) → 2 → 1 → 0 순서로 변한다. 25~45 ns에 4가 두 클럭 유지되는 것은 26 ns에 enable=0이 되어 35 ns 에지를 건너뛰었기 때문이며, 이때 serial_in=1이 들어와 있어도 값이 바뀌지 않는다. 비트 파형에서 1이 value[3]→[2]→[1]→[0]으로 한 클럭씩 대각선으로 내려가는 모양이 보이며, 이는 {serial_in, value[3:1]} 연결식이 이전 에지의 단 값을 한 칸씩 넘

긴다는 뜻이다. 0을 계속 넣으면 85 ns에 모든 비트가 0으로 비워지고, 마지막 rst는 serial_in=1보다 우선한다.

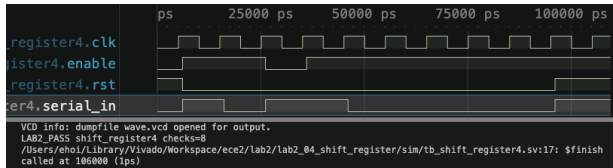


그림 8. Lab 2.04 VS Code 파형과 Icarus 로그. LAB2_PASS shift_register4 checks=8, \$finish 106000 ps.

비교. VS Code 파형의 enable 휴지 구간(26~36 ns)과 serial_in 1:0 패턴이 Vivado와 같은 시각에 나타나며, 두 도구 모두 106 ns에서 8개 검사를 마쳤다.

3.5 Lab 2.05 PISO

1010을 병렬 로드하면 직렬 출력은 최상위 비트 1부터 1, 0, 1, 0 순서로 나오고, 네 번 시프트한 뒤 저장값은 0000이다. load와 enable이 동시에 1이면 load가 우선한다 [1, p.18].

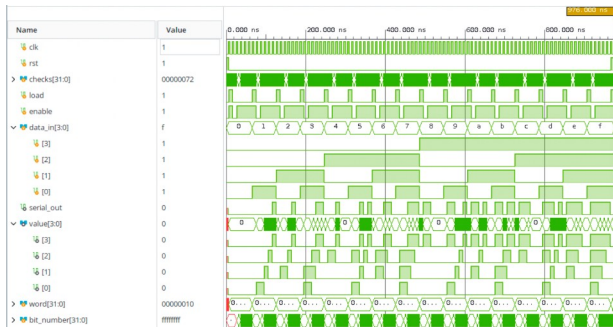


그림 9. Lab 2.05 Vivado XSim 파형. load-enable, data_in(0~f), serial_out, value와 checks=0x72(114).

해석. TB는 word 0부터 f까지 16개 값을 60 ns 간격으로 로드한다. 각 구간은 load-enable 동시 1(로드 우선) 1클럭, hold 1클럭, 시프트 4클럭으로 구성된다. data_in이 8(1000) 이상인 구간에서는 value[3]이 로드 직후 1이 되고, serial_out은 value[3]을 조합으로 내보내므로 예지 전에 이미 MSB가 출력된다. 이후 매 예지 {value[2:0], 1'b0} 시프트로 다음 비트가 MSB로 올라오고 LSB는 0으로 채워진다. 예를 들어 a(1010) 구간의 serial_out은 1, 0, 1, 0이고, 네 번째 시프트 뒤 value는 0이다. 16개 값 × 7개 검사 + 리셋 검사 2개 = 114개가 976 ns에 끝났다.

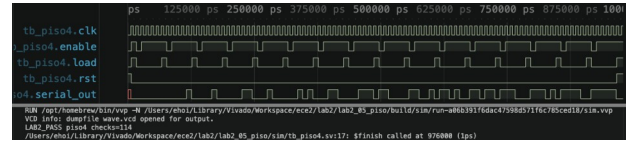


그림 10. Lab 2.05 VS Code 파형과 Icarus 로그. LAB2_PASS piso4 checks=114, \$finish 976000 ps.

비교. VS Code의 serial_out 펄스 열은 word가 커질수록 1 비트가 많아지는 같은 패턴을 보이며, load-enable 펄스 간격도 Vivado와 같다.

3.6 Lab 2.06 Moore 상태 머신

상태는 00→01→10→00으로 순환하며 enable과 advance가 모두 1일 때만 전이한다. 출력은 상태 자체이므로 클럭 사이에 advance만 바뀌어도 출력은 바뀌지 않아야 한다 [1, p.19].

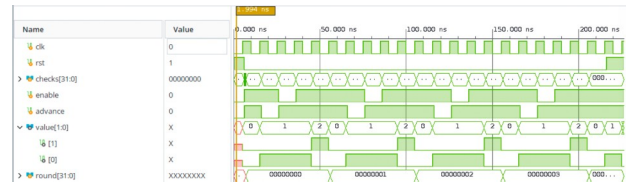


그림 11. Lab 2.06 Vivado XSim 파형. enable-advance와 value[1:0], round. 커서는 1.994 ns에 있으며, 원본 캡처의 Objects 창은 checks=23을 표시한다.

해석. 6 ns에 advance=1이 되어도 value는 15 ns에 지까지 0이다(출력이 입력만으로 바뀌지 않음). 각 round에서 value는 1이 된 뒤 세 클럭 동안 유지된다. 16 ns에는 advance=0, 26 ns에는 enable=0이어서 두 조건 중 하나라도 0이면 전이가 막히기 때문이다. 36 ns에 두 신호가 다시 1이 되면 45 ns에 2, 55 ns에 0으로 전이한다. 파형의 0 1 2 0 반복이 4 round 이어지고, 한 번 더 S1로 전이한 뒤 마지막 rst로 0이 되어 226 ns에 23개 검사가 끝난다. value의 모든 변화가 clk 상승 에지에만 정렬되어 있는 것이 Moore 출력의 특징이다.



그림 12. Lab 2.06 VS Code 파형과 Icarus 로그. LAB2_PASS moore_cycle checks=23, \$finish 226000 ps.

비교. VS Code 파형의 advance-enable 휴지 패턴은 round마다 같은 모양으로 네 번 반복되며, Vivado의 value가 1로 멈춰 있는 구간과 정확히 겹친다.

3.7 Lab 2.07 Mealy 상태 머신

S0에서 입력 1이면 출력 10, S1에서 입력 1이면 01, 입력 0이면 00이다. 유효 에지에서 입력이 1이면 S0→S1로 전이하고, enable=0이어도 입력에 따른 조합 출력은 계속된다 [1, p.20].

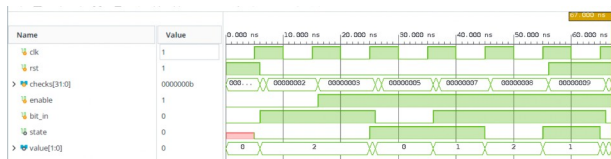


그림 13. Lab 2.07 Vivado XSim 파형. enable-bit_in, state, value[1:0]와 checks=0xb(11).

해석. 6 ns에 bit_in이 1이 되면 에지를 기다리지 않고 value가 즉시 2(10)가 된다. 15 ns 에지에서는 enable=0이라 state가 유지되지만 value=2도 그대로다(비활성 중에도 Mealy 출력 존재). 25 ns 에지에서 state가 1로 토글하면 value는 1(01)로 바뀌고, 26 ns에 bit_in이 0이 되자 value가 곧바로 0이 된다. 36 ns에 bit_in=1이 되면 에지 전인데도 value=1이 나타나고, 45 ns~55 ns 에지에서 state가 0, 1로 토글하며 value도 2, 1로 따라간다. 즉 value의 변화 시점 가운데 6, 26, 36 ns는 클럭 에지가 아닌 입력 변화 시점이며, 이것이 3.6절 Moore 파형과 구별되는 점이다.

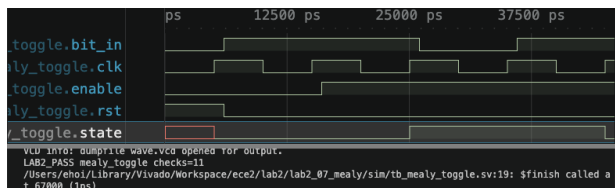


그림 14. Lab 2.07 VS Code 파형과 Icarus 로그. LAB2_PASS mealy_toggle checks=11, \$finish 67000 ps.

비교. VS Code 파형에서 state는 25 ns와 45 ns 부근 에지에서만 바뀌고, bit_in은 6 ns와 26 ns, 36 ns에 바뀐다. state와 입력 변화 시점이 Vivado와 같으므로, 두 도구가 조합 출력의 즉시 반응을 같게 계산했음을 확인하였다.

3.8 Lab 2.08 7세그먼트 자리 스캔

8자리 중 한 자리만 켜고 순서대로 반복하며, 자리 전환 사이에는 모든 자리 선택을 0으로 하는 blank 구간을 둔다. 기능 코어는 active-high 논리값을 내고, 실제 보드의 극성은 top에서 맞춘다 [1, p.21].

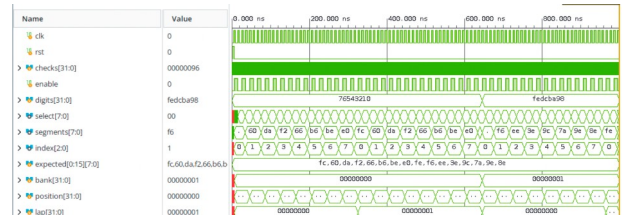


그림 15. Lab 2.08 Vivado XSim 파형(0~1000 ns). digits, select, segments, index, bank-lap-position과 checks=0x96(150).

해석. 0~646 ns(bank 0)에는 digits=76543210, 이후 (bank 1)에는 fedcba98이 입력된다. index는 0→7을 한 바퀴에 320 ns씩 돌고 bank마다 두 바퀴(lap 0, 1)를 반복한다. TB는 enable을 매 클럭 토글하므로 한 자리당 4클럭(선택 2클럭, blank 2클럭)이 걸린다. select는 one-hot 값과 00을 번갈아 가지며 00인 구간이 blank이다. segments는 bank 0에서 fc, 60, da, f2, 66, b6, be, e0(0~7), bank 1에서 fe, f6, ee, 3e, 9c, 7a, 9e, 8e(8~f)로 기대값 표 expected[0:15]와 같다.



그림 16. Lab 2.08 Vivado 파형의 bank 0 첫 바퀴(0~330 ns) 확대. index 0~7에 대응하는 segments가 fc, 60, da, f2, 66, b6, be, e0 순서로 나타난다.

확대 파형에서 segments는 index가 바뀌는 에지, 즉 select가 00이 되는 blank 시작 시점에 이미 다음 자리의 코드로 바뀐다. 다음 자리의 데이터가 blank 동안 먼저 안정된 뒤 선택 신호가 켜지므로, 자리 전환 순간 이전 숫자가 다음 자리에 비치는 잔상을 막을 수 있다. 이 관계는 6장의 보드 관찰을 해석하는 기준이 된다.

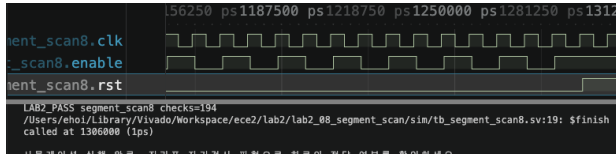


그림 17. Lab 2.08 VS Code 파형 (종료부)과 Icarus 로그. LAB2_PASS segment_scan8 checks=194, \$finish 1306000 ps.

비교. VS Code 캡처는 마지막 구간에서 enable이 매 클럭 토글하고 1296 ns에 rst가 올라가는 모습을 보여 준다. 194개 검사가 1306 ns에 끝났으며, 이는 $1 + 32 \text{ 자리} \times 6 \text{ 개} + 1 = 194$ 와 일치한다. Vivado 캡처의 checks=150은 1000 ns 시점까지의 진행 수로, 같은 시점의 Icarus 진행과 모순되지 않는다.

4 수정실험 결과

교안은 회로마다 RTL 한 곳을 바꾸고 TB 기대값은 유지하여 자기검사가 어떤 조건에서 실패하는지 확인 하도록 한다. 8건 모두 교안이 예고한 검사 이름과 시각에서 처음 실패했고, 원복 후 정상 실행과 같은 검사 수·종료 시각으로 다시 통과하였다(표 5).

표 5. 수정실험의 변경 내용과 첫 실패 지점. 복구 후 8건 모두 표 3과 같은 검사 수로 LAB2_PASS.

실 습	RTL 변경	첫 실패 검사 · 시각	기대 / 실제
01	증가량 +1 → +2	up including 15 to 0 · 16 ns	1 / 2
02	상승 조건 DIVISOR/2-1 → DIVISOR/2	minimum divisor duty · 16 ns	div2 1 / 0
03	value <= stored → data_in	transfer stored not live input · 26 ns	aa / a3
04	이동 방향 반전	input enters MSB · 16 ns	1000 / 0001
05	serial_out = value[0]	MSB first before edge · 86 ns	0 / 1
06	S1의 다음 상태 10 → 00	S1 to S2 · 46 ns	10 / 00
07	입력 1의 출력 10↔01 교환	S0 input changes between clocks · 7 ns	010 / 001
08	blank 조건 제거	reset blanks digit zero · 6 ns	select 00 / 01

실패 시각은 모두 "그 시각 직전의 입력과 이전 상태"로 설명된다. 01은 리셋 뒤 첫 증가 예지(15 ns)에서 $0+2=2$ 가 되어 16 ns 검사에서 걸렸다. 03은 25 ns 예지에서 value가 저장값 a 대신 현재 입력 3을 받아, 3.3

절에서 해석한 "전달은 저장값을 사용한다"는 조건이 TB에 반영되어 있음을 보여 준다. 04와 05는 비트 순서 오류가 첫 입력 또는 첫 1비트 출력에서 바로 드러났고, 06은 S1에서 S2로 가는 첫 전이, 07은 예지 없이 입력만 바뀐 7 ns의 조합 출력에서 검출되었다. 08은 리셋 중에도 index=0 자리가 선택되어 blank가 없으면 첫 검사부터 실패한다.

Lab 2.01 수정실험은 00 START 교안 [1, p.14]에만 지시가 있어 기존 기록에서 누락되어 있었다. 보고서 작성 과정에서 이를 확인하고 2026-09-28에 Icarus 12.0으로 정상 RTL은 그대로 둔 채 증가식만 바꾼 사본을 실행하여 로그와 VCD를 evidence/modified-recovered에 보관하였다.

5 합성·구현과 bit 파일 (Lab 2.08)

5.1 설계 계층과 핀·타이밍 조건

설계 파일은 segment_scan8.v(스캔 코어), input_frontend.v(입력 동기화·디바운스), lab2_segment_scan.v(보드 top)이다. 시뮬레이션 top은 tb_segment_scan8, 합성 top은 lab2_segment_scan이다. top은 코어의 enable을 1로 고정하고 표시값을 $\text{digits} = \{28'h7654321, \text{sw}[7:4]\}$ 로 만든다. 즉 첫 자리(index 0)는 DIP 스위치 상위 4비트, 나머지 일곱 자리는 1~7로 고정된다. 자리 선택은 보드가 active-low이므로 반전하고, index 0을 COM[7]에 연결한다.

표 6. 핀 배치(모든 I/O는 LVCMOS33). LED-DIP-KEY의 대응은 Lab 1과 같은 보드 핀 정의를 따른다 [3].

포트	핀	보드 장치 · 용도
clk	B6	주 클록, 1 kHz로 설정
rst	K4	KEY1, 비동기 인가·동기 해제 리셋
button	N8	KEY2, 동기화만 되고 표시에는 미사용
sw[7:4]	Y1, W3, U2, T1	DIP1~4, 첫 자리 값
sw[3:0]	W4, W1, V4, U4	DIP5~8, 미사용
led[2:0]	M3, M1, N5	LED6~8 = index[2:0]
led[7:3]	L4, M4, M2, N7, M7	LED1~5, 0 고정
seg_data[7:0]	F1 ... H2	세그먼트 a~g, dp (active-high)
seg_com[7:0]	H4 ... K5	자리 선택 (active-low)

타이밍 제약은 `create_clock -period 1000000.000(1 kHz, 1 ms)`이며, 비동기 입력인 `rst`, `button`, `sw[*]`에는 `set_false_path`를 적용하였다. 이 입력들은 내부에서 2단 동기화되므로 외부 입력 경로를 클록 경로로 분석하지 않도록 한 것이다.

5.2 자원 사용량과 타이밍

표 7. Lab 2.08 배치·배선 후 결과(Vivado 2026.1) [2].

항목	구현 결과	해석
Slice LUT	14 / 48,000 (0.03%)	디코더·선택 논리
Slice Register	14 / 96,000 (0.01%)	index-blank·동기화 FF
Bonded IOB	30 / 338 (8.88%)	입력 6 + 출력 24
BUFGCTRL	1	clk 전역 버퍼
WNS / TNS	999998.062 ns / 0 ns	setup 위반 0
WHS / THS	0.119 ns / 0 ns	hold 위반 0

합성 직후에는 LUT 12·레지스터 24·IOB 35개였으나 배치 후 LUT 14·레지스터 14·IOB 30개로 집계되었다. 합성 단계의 IOB 35개는 입력 11개(`clk`, `rst`, `button`, `sw` 8)와 출력 24개의 합이며, 배치 후 5개가 줄어든 것은 표시에 쓰이지 않는 `button`과 `sw[3:0]`이 최적화로 제거되었기 때문이다. 남은 레지스터 14개도 리셋 동기화 2 + `sw[7:4]` 2단 동기화 8 + `index` 3 + `blank` 1로 정확히 설명되며, `press`를 만드는 디바운스 카운터 역시 출력이 없어 제거되었다. WNS가 약 1 ms인 것은 클록 주기가 1,000,000 ns인 데 비해 실제 경로 지연이 약 2

ns에 불과하기 때문이며, 회로의 고속 동작 한계를 뜻하지 않는다.

5.3 경고 해석과 bit 파일

최종 DRC는 오류 0건이다. **TIMING-18** 경고 24건은 입출력 delay 제약이 없다는 경고인데, 개수가 출력 포트 수(`led` 8 + `seg_data` 8 + `seg_com` 8 = 24)와 같다. 입력은 `false path`로 제외했으므로 출력 포트에 `set_output_delay`가 없다는 뜻으로 해석된다. LED와 7 세그먼트는 사람이 보는 ms 단위 표시 장치이므로 기능에는 영향이 없다. **CFGBVS-1** 경고 1건은 구성 बैं크 전압 속성이 없다는 것으로, XDC에 `CFGBVS VCC0`와 `CONFIG_VOLTAGE 3.3`을 추가하면 해소된다.

생성된 파일은 `lab2_segment_scan.bit(3,687,020 byte)`이며 SHA-256은 `abe37e050585098e9810a80b29096d238dc6ca89a6e9c7c5e cee5f68dd2be47`이다. 구현 경로는 `lab2_08_segment_scan/vivado/vivado.runs/impl_1`이다.

6 보드 시현 결과와 시뮬레이션 비교

6.1 시뮬레이션과 보드의 시간 척도

3.8절의 시뮬레이션과 보드 동작은 같은 코어를 쓰지만 클록과 `enable` 조건이 다르다. 표 8처럼 보드에서는 `enable`이 1로 고정되어 한 자리가 선택 1클록, `blank` 1클록을 차지하므로 8자리 한 바퀴가 16 ms, 화면 갱신률은 62.5 Hz이다. 이는 눈의 잔상 효과로 깜빡임 없이 보이는 범위이다. 각 자리는 16 ms 중 1 ms만 켜지므로 여덟 자리가 "동시에" 보이는 것은 빠른 순환의 결과이다.

표 8. 시뮬레이션과 보드의 스캔 시간 비교.

항목	XSim-Icarus	HBE 보드
clk 주기	10 ns	1 ms (1 kHz)
enable	매 클럭 토글(TB)	1 고정
한 자리 선택 / blank	20 ns / 20 ns	1 ms / 1 ms
8자리 한 바퀴	320 ns	16 ms (62.5 Hz)
자리당 점등 비율	1/16	1/16 (6.25%)
digits	76543210 / fedcba98	7654321 + sw[7:4]

6.2 사진별 관찰과 파형 대응

아래 네 장은 시현 영상에서 DIP 스위치를 차례로 올리며 캡처한 장면이다. 두 번째 자리부터는 모든 사진에서 1~7이 유지되고 첫 자리만 바뀌므로, 첫 자리의 세그먼트를 3.8절의 코드와 비교하였다.

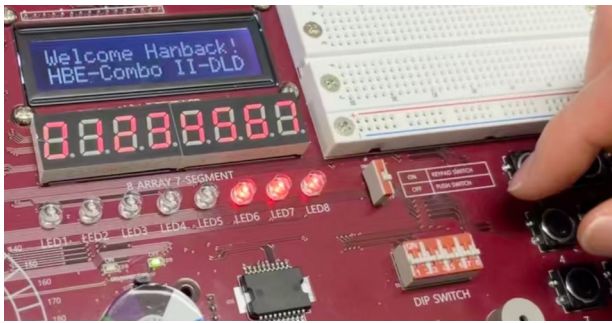


그림 18. (a) DIP1~4 모두 OFF(표시값 기준). 8자리에 01234567이 표시되고 LED6~8이 켜져 있다. 오른쪽은 키패드 버튼을 누르는 손이다.

(a) 기준 상태. sw[7:4]=0000 이므로 digits는 76543210이 되어 시뮬레이션 bank 0과 완전히 같다. 왼쪽부터 0, 1, ..., 7은 그림 16의 segments fc, 60, da, f2, 66, b6, be, e0에 해당한다. index 0이 COM[7]에 연결되어 가장 왼쪽 자리에 나타나고, index가 증가할수록 오른쪽으로 이동한다. dp는 코어가 항상 0을 내므로 모든 자리에서 꺼져 있다. 사진에서 버튼을 누르는 중에도 표시가 그대로인 것은, button 입력이 동기화 디바운스만 되고 press 출력이 코어에 연결되지 않은 top 구조와 모순되지 않는다.

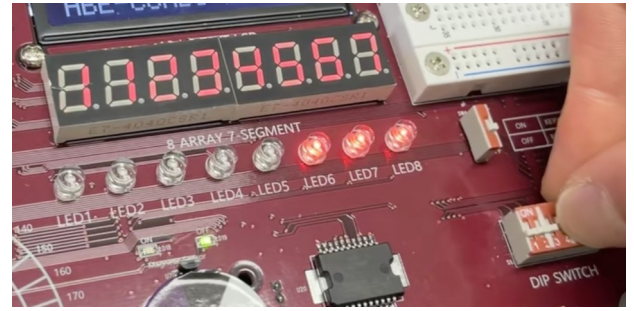


그림 19. (b) DIP4를 켜는 장면. 첫 자리가 1로 바뀌었다.

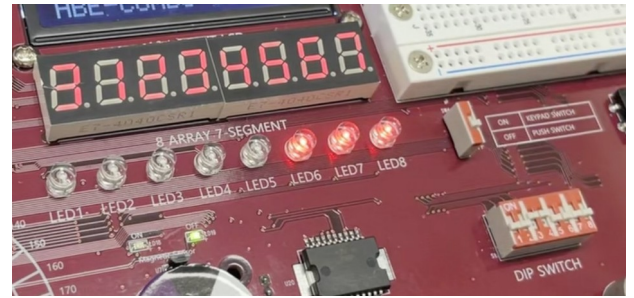


그림 20. (c) DIP3-DIP4 ON. 첫 자리가 3이다.

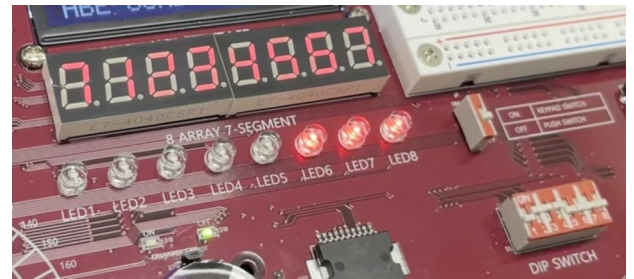


그림 21. (d) DIP2~4 ON. 첫 자리가 7이다.

(b)~(d) 입력 변화. DIP1~4는 sw[7:4]에 순서대로 연결되어 있으므로(표 6), DIP4, DIP3, DIP2를 차례로 켜면 첫 자리 값은 0001, 0011, 0111, 즉 1, 3, 7이 된다. 사진의 첫 자리는 각각 b-c 세그먼트(1), a-b-c-d-g(3), a-b-c(7)가 켜져 있으며, 이는 시뮬레이션 디코더의 60, f2, e0과 정확히 같다(표 9). 스위치 변화는 2단 동기화(2 ms) 뒤 index 0이 다시 선택될 때(최대 16 ms) 반영되므로 사람이 느끼기에는 즉시 바뀐다.

표 9. 보드 첫 자리 표시와 시뮬레이션 세그먼트 코드의 대응. sw[7:4]는 표시값과 DIP 조작 장면으로부터 정한 값이다.

사진	sw[7:4]	첫 자리	segments	켜진 세그먼트
(a)	0000	0	fc	a b c d e f
(b)	0001	1	60	b c
(c)	0011	3	f2	a b c d g
(d)	0111	7	e0	a b c

6.3 LED 출력의 해석

top은 `led = {5'b00000, index}`이므로 LED6-7-8은 `index[2]:[1]:[0]`을, LED1-5는 항상 0을 출력한다. 모든 사진에서 LED1-5는 꺼져 있고 LED6-8만 켜져 있어 핀 배치와 일치한다. `index`는 2 ms마다 증가하므로 `index[0]`은 250 Hz, `index[1]`은 125 Hz, `index[2]`는 62.5 Hz의 50% 듀티 구형파이다. 모두 잔상 한계보다 빨라 세 LED가 깜빡임 없이 절반 밝기로 계속 켜진 것처럼 보인다. 시뮬레이션에서 `index`가 0-7을 반복하던 파형이 보드에서는 세 LED의 "항상 켜짐"으로 관찰되는 것이다.

정지 사진은 한 순간만 담으므로 blank 구간이나 자리 선택 순서를 직접 측정한 것은 아니다. 여러 자리가 동시에 켜져 보이는 것, 인접 자리에 이전 숫자의 잔상이 보이지 않는 것, 입력 변화가 첫 자리에만 반영되는 것이 3.8절의 파형 해석과 모두 일치한다는 점을 근거로 판단하였다. 스위치 조작과 표시 변화의 연속 과정은 시현 영상(9장)에 기록되어 있다.

7 고찰

7.1 Moore와 Mealy 출력의 시점

Moore 파형(그림 11)에서 `value`의 모든 변화는 `clk` 상승 에지에 정렬되었고, `advance`가 먼저 바뀌어도 다음 에지까지 출력이 유지되었다. 반면 Mealy 파형(그림 13)에서는 `bit_in`이 바뀐 6, 26, 36 ns에 에지 없이 `value`가 바뀌었다. Mealy는 입력에 같은 클록 안에서 반응하는 대신 입력의 글리치가 출력으로 전달될 수 있고, Moore는 한 클록 늦지만 출력이 레지스터에서 나와 안정적이다. 수정실험 07이 7 ns의 클록 사이 검사에서 실패한 것도 Mealy 출력이 입력의 조합 함수라는 점을 TB가 직접 검사하기 때문이다.

7.2 nonblocking 대입과 동시 제어

레지스터 쌍의 35 ns 에지(그림 5)는 `load`와 `transfer`가 동시에 1일 때 `value`가 새 입력이 아닌 이전 `stored`를 받는 것을 보여 주고, 시프트 레지스터의 대각선 비트 이동(그림 7)도 같은 원리이다. 같은 에지에서 모든

레지스터가 갱신 전의 값을 읽기 때문에 파이프라인처럼 한 단씩 전달된다. 만약 `blocking` 대입을 쓰면 문장 순서에 따라 값이 한 에지에 두 단을 건너뛸 수 있다. 수정실험 03과 04는 이 전달 경로를 바꾸었을 때 TB가 첫 전달에서 바로 검출함을 확인시켜 주었다.

7.3 blank 구간의 역할과 비용

`blank`는 `index`와 `segments`가 바뀌는 순간 `select`를 0으로 만들어 이전 자리의 데이터가 다음 자리에 비치는 잔상을 막는다(그림 16). 대신 자리당 점등 비율이 `blank`가 없을 때의 1/8에서 1/16으로 줄어 밝기가 낮아진다. 보드 사진에서 잔상 없이 숫자가 또렷하게 구분된 것은 `blank`의 효과로 볼 수 있다. 더 밝은 표시가 필요하면 `blank`를 1클록으로 유지하면서 선택 구간을 여러 클록으로 늘려 비율을 높일 수 있다.

7.4 검증 단계별 근거와 한계

Icarus PASS는 TB가 정의한 조건을 통과했다는 뜻이고, Vivado 파형은 다른 시뮬레이터에서도 같은 전이가 일어남을 보여 준다. 구현 리포트는 회로가 1 kHz 제약 아래 배치·배선되었음을, 보드 사진은 물리 출력이 파형 해석대로 나타남을 보여 준다. 다만 Lab 2.08의 Vivado 캡처는 1000 ns(150건) 시점이므로 XSim의 194건 완료는 캡처로 확인하지 못했다. 출력 `delay`와 구성 전압 경고가 남아 있으며, Hardware Manager의 장치 프로그램 완료 화면은 보존되지 않아 첨부하지 못했다.

8 결론

8개 순차논리회로를 Icarus와 Vivado XSim에서 실행하여 모두 같은 검사 수와 종료 시각으로 통과함을 확인하고, 각 파형에서 리셋 우선, `enable` 유지, `nonblocking` 전달, Moore-Mealy 출력 시점, 스캔과 `blank` 동작을 해석하였다. 교안의 수정실험 8건은 예고된 검사와 시각에서 실패하고 복구 후 통과하여 TB가 설계 의도를 검출함을 보였다. Lab 2.08은 LUT 14개, 레지스터 14개로 구현되어 `setup-hold` 위반 없이 bit 파일을 생성하였고, 보드에서 DIP 조작에 따라 첫 자리가 0→1→3→7로 바뀌는 모습이 시뮬레이션의 세그먼트

트 코드 fc60f2e0과 일치하였다. LED6-8의 상시 점등은 2 ms마다 바뀌는 index가 잔상 효과로 합쳐져 보인 결과로 설명된다.

9 제출 자료와 참고문헌

GitHub 저장소.

https://github.com/tae2yi/UOS_ece2/tree/master/lab2

기준 커밋 ID.

[27eefdb44945af11842fe104defd1358e2aa2b1b](#) (master, "lab2: 실험 01-08 프로젝트 소스 추가"). RTL·TB·XDC와 정상·수정·복구 로그는 이 커밋 기준이며, Lab 2.01 수정실험 증빙과 본 보고서는 이후 커밋에 추가하였다.

보드 시현 영상. <https://youtube.com/playlist?list=PLbHILnDMwa6Y&si=6yP5VxLgyYnV-kqB> (YouTube 재생목록 "ece2-lab2")

자료 위치. RTL·TB·XDC와 로그: lab2/lab2_01_counter ... lab2_08_segment_scan. VS Code 캡처: lab2/images. Vivado 캡처와 보드 사진: lab2/reports/final/images. 보고서 그림 생성: lab2/reports/final/make_figures.py.

- [1] 이해리, "LAB 2 · 순차논리 예습", 전기전자컴퓨터설계실험 2, 05.LAB2_00_START, 2026-09-10, pp.5-25.
- [2] Lab 2.01-2.08 Vivado 교안(05.LAB2_0x*_VIVADO)의 수정실험 쪽: 02 p.44, 03 p.43, 04 p.41, 05 pp.41-43, 06 pp.39-41, 07 pp.41-43, 08 pp.48-50; 08 구현 pp.77-81. Lab 2.08 README, XDC, simulation.log, Vivado utilization-timing-DRC 리포트(2026-09-21).
- [3] 김태이, LAB 1 결과보고서, 2026-09-20. KEY:DIP-LED 핀 대응.